# BriskSnapshot: A Live Demo of Join-Backed Semantic Windows for Streaming AI Pipelines

Anonymous Author(s)

## Abstract

Streaming AI applications need to keep vector state fresh while exposing that state in a form that downstream services can inspect and act on. BriskSnapshot is a live demonstration of a unified semantic-window system implemented over a streaming AI control layer and a vector-stream runtime [2]. The demo follows public vulnerability-intelligence reports as they are normalized, embedded, joined inside a bounded semantic window, and surfaced as snapshot contracts for a provider-agnostic generator service. Unlike a static retrieval demo, BriskSnapshot lets the audience configure the stream, join strategy, retention window, parallelism, and generator backend while seeing how those choices affect runtime evidence and downstream answers. The live walkthrough stays small enough for narration, while a 3,000-record NVD replay and a 20-answer generator run show measured behavior across the vector runtime and contract-to-generation path.

## CCS Concepts

• **Computer systems organization** → **Parallel architectures**; • **Information systems** → **Stream management**; • **Computing methodologies** → *Artificial intelligence.*

## Keywords

stream processing, vector search, AI middleware, semantic windows, demonstration

## 1 Introduction

Large-model applications increasingly run as long-lived dataflows rather than as isolated prompt calls. A vulnerability assistant receives new advisories while an investigation is still open, and a retrieval pipeline refreshes its vector neighborhood between two user questions. In these settings, the model-facing component should not receive an opaque vector index or a free-form prompt dump. It needs bounded, inspectable evidence: which records are retained, which records matched, which sources support the cluster, and which action the system recommends.

Stream processors provide scalable dataflow execution [1, 3], while vector indexes provide efficient approximate search [5, 6, 9]. Retrieval-augmented systems further show how language models benefit from external evidence [4, 7], and recent work brings retrieval into streaming settings, including vector-native dataflow engines [8] and streaming RAG frameworks that update knowledge contexts as events arrive [10]. What is still awkward in many prototypes is the boundary between streaming state and AI orchestration. Semantic state is often rebuilt outside the component that observes the stream, so a demo can show an answer but not the live runtime evidence that made the answer possible.

BriskSnapshot demonstrates a narrower and more observable boundary. The system owns the active semantic window, runs vector joins as events arrive, and exports only compact contracts to the application layer. A contract is not another prompt; it is a structured object with cluster identity, supporting neighbors, source coverage, novelty, route, priority, and action fields. Those fields are then rendered in the UI and placed into the generator context, so the audience can trace an answer back to runtime-owned evidence rather than to hidden prompt construction. The demo is built around the question an audience naturally asks during a live system demonstration: when the next event arrives, what changes, why did it change, and what can the upper pipeline safely do with the change?

Compared with prior descriptions of the same demo, the paper sharpens three aspects: the system architecture, the live console, and the prepared evidence. The architecture is described as a single integrated prototype rather than two adjacent components. The live console itself becomes part of the contribution: attendees can zoom from the end-to-end application pipeline into the semantic-window operator path, inspect aggregated evidence, and verify the contract consumed by the generator. The prepared evidence draws on real public records and real embedding vectors, including a parallelism sweep and a generator run, to show the capability of the prototype without turning the paper into a full benchmark paper.[1]

## 2 System Overview

Figure 1 shows the implemented path. BriskSnapshot takes heterogeneous vulnerability reports from multiple sources, maps each report into a dense text embedding, appends the record into a bounded semantic window, and drains join pairs produced by the persistent runtime. The output is then folded into application-level contracts and passed to a provider-agnostic generator service through a bounded evidence context. This organization is intentionally close to the live demo: the top-level event-to-answer pipeline stays visible while the semantic-window stage expands into append lanes, window state, join strategy, pair emission, cluster update, and snapshot export. The audience can therefore follow the same event from stream input to runtime-owned evidence to generated answer.

### 2.1 Design Goals

Three design goals shape what the demo is supposed to make visible: whether retained context is current, what the application is allowed to see, and how others can reproduce the same observation off-line.

*Observable freshness.* A static retrieval index can demonstrate relevance, but it hides whether the context is current. BriskSnapshot keeps the current window visible: retained records, emitted pairs, active clusters, and newest evidence are all displayed as runtime state rather than as post-hoc explanation text.

*Bounded handoff.* Model-facing services should not receive an unbounded list of nearest neighbors. The system exports a small set of fields that can be logged, routed, or summarized: cluster

[1] Anonymous source for review: https://anonymous.4open.science/r/SAGE-7113, https://anonymous.4open.science/r/sageFlow-5772.

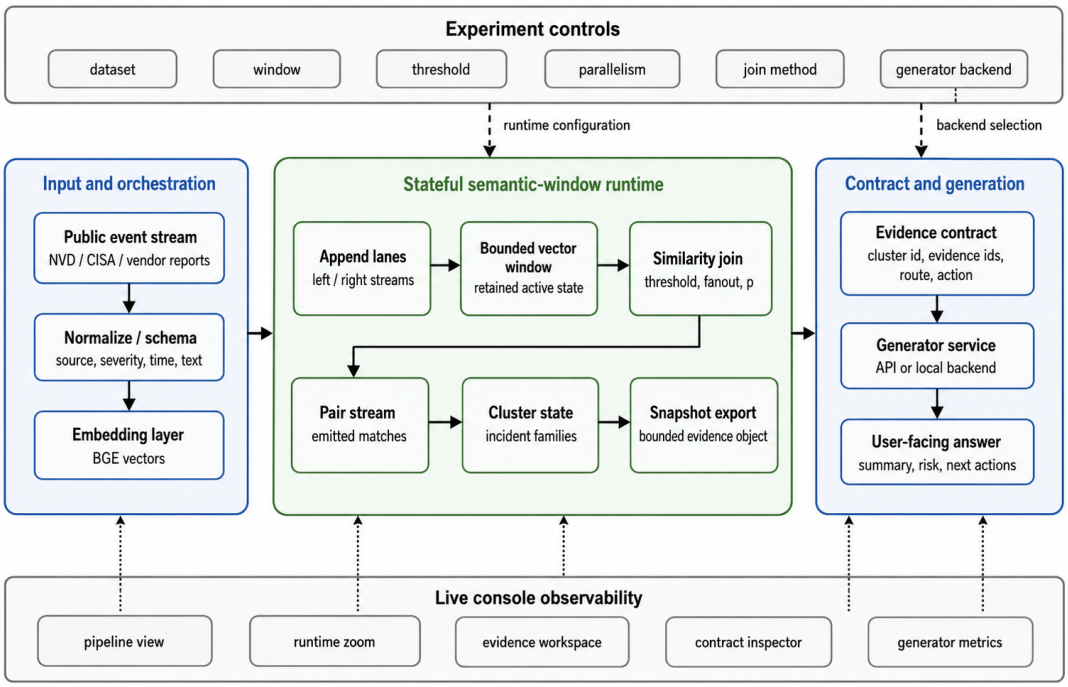


**Figure 1: BriskSnapshot architecture. Experiment controls configure the stream, runtime, and generator; the semantic-window runtime owns retained vector state, joins, pair emission, cluster state, and snapshot export; the contract/generation layer consumes bounded evidence for traceable answers.**

identity, source coverage, supporting neighbor identifiers, novelty, route, priority, and action. This turns the semantic window into an application interface rather than a private runtime detail.

*Demo reproducibility.* Live demos are convincing when they are interactive, but papers also need evidence that the observed behavior is not a hand-picked screen. The same replay driver used for the UI can run public NVD/CISA inputs, cached vector artifacts, threshold variants, and window-size variants. The prepared figure in Section 4 is generated from those replay artifacts.

## 2.2 Runtime Boundary

The runtime boundary is deliberately narrow. The ingestion layer gives the runtime comparable vectors and stable identifiers. The window manager owns which events are still active and which events have left the current view. The join runtime is the only component that creates semantic match pairs. The contract layer interprets those pairs for an application: a dense multi-source cluster becomes an incident snapshot, a cluster with a clear owner becomes a routing contract, and a high-severity novel event becomes an escalation contract. The UI exposes the same boundary through the pipeline view, runtime zoom, evidence tables, contract inspector, and generator metrics.

Cluster construction uses connected components over returned join pairs. Clusters are sorted by size and average severity before export, which makes the display stable enough for a presenter to narrate. The default incident rule requires at least three events, average severity above 0.70, and at least two source classes. The emerging-risk rule fires when a high-severity event remains novel enough to deserve attention even before a larger cluster forms.

# 3 Live Demonstration

The live scenario uses public vulnerability-intelligence records from NVD, vendor advisories, CISA warnings, emergency alerts, and research notes.[2] The reports cover familiar incident families such as PAN-OS, XZ Utils, OpenSSH, PHP-CGI, Fortinet VPN, and TeamCity. This domain is useful for a systems demo because records arrive from different sources, use different language, and still need to be grouped quickly enough for operator-facing workflows. The console is therefore organized as an experiment surface: the presenter can choose replay source, join method, window size, threshold, parallelism, and generator backend before following the same configured run through pipeline state, runtime operators, evidence, and generated output. The larger evidence run uses 3,000 NVD records published in the first quarter of 2024, embedded once with BGE and replayed through the same runtime path.

## 3.1 Audience Path

Figure 2 shows the UI used in the demonstration. Its role is not to be a dashboard after the fact, but to make the experiment protocol visible while the pipeline runs. The top surface fixes the workload and runtime parameters. The pipeline view keeps the event-to-answer

[2] A short demonstration video is available for review at https://www.dropbox.com/scl/fi/n3gn14boswpbjqhxazlb2/brisksnapshot-demo.mp4?rlkey=nn2m6cxdekxdxhvikvst7p4bl&dl=0.

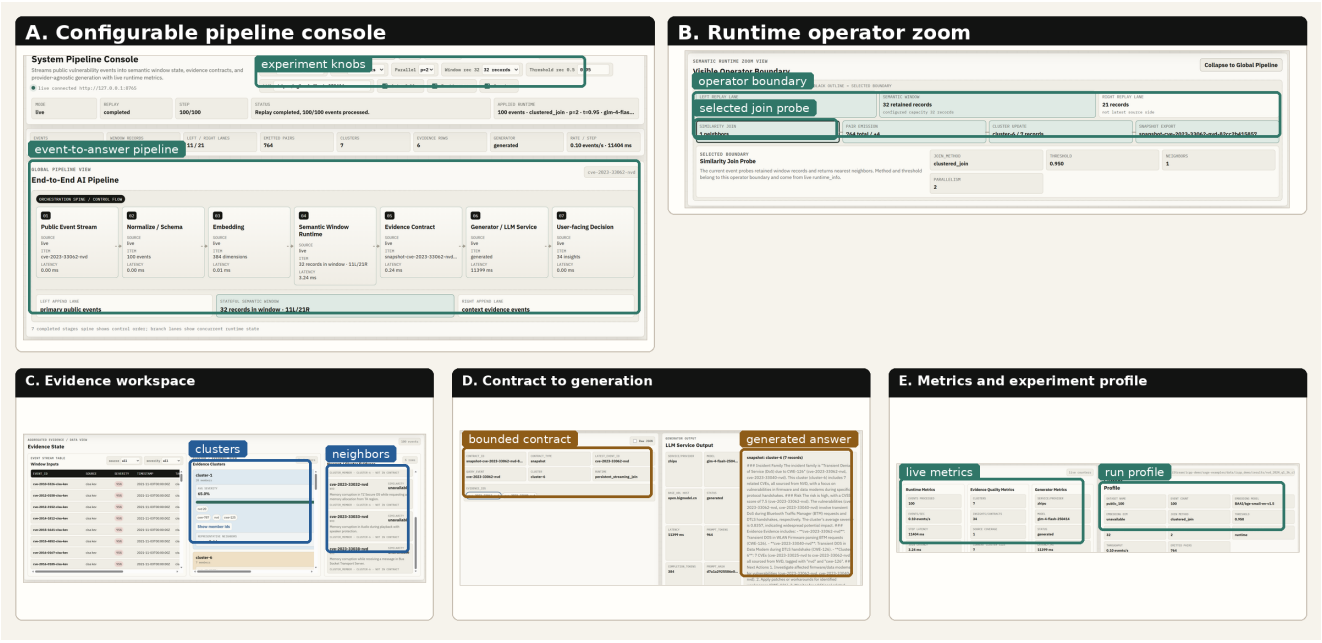


**Figure 2: Real console walkthrough. Annotated screenshots show the configurable pipeline console, semantic-window operator zoom, evidence workspace, contract-to-generation handoff, and metrics/profile view used during the live demonstration.**

path in one place. The zoom view opens the semantic-window stage into the operators that actually create evidence: append lanes, retained state, join probe, pair stream, cluster update, and snapshot export. The lower surfaces then carry that evidence into rows, clusters, neighbor records, a bounded contract, and provider-agnostic generation.

The presenter can run the stream stepwise or as a replay, change configuration, and then inspect how the same pipeline responds. This is the main demonstration point: parameter changes are not frontend decoration. They alter the boundary between retained runtime state, emitted semantic matches, exported contracts, and the text produced by the generator. The metrics/profile view records the selected workload, join method, threshold, window, parallelism, and generator backend next to runtime, evidence, and generation counters.

The walkthrough follows a concrete event from arrival to generated answer. The presenter first points at what just arrived in the stream, then traces it through the pipeline, opens the runtime view to show what was retained and what was joined, walks through the records that became evidence, and finally compares the contract handed to the generator with the answer the generator produced. This is the same structure used in many systems demonstrations: start with a concrete user path, reveal internal state at the moment it matters, and close by showing measured evidence beyond the exact path shown on screen.

## 3.2 Demonstration Scenarios

The live demonstration unfolds as three scenarios that follow the same event from snapshot, into the runtime, and out to the generator.

**Table 1: Default live-demo configuration and interactive controls.**

| Control | Default | What changes in the demo |
|---|---|---|
| Replay source | 100 events | Stream length and visible event path |
| Join method | clustered join | Join probe and emitted-pair behavior |
| Window | 32 records | Retention horizon and cluster timing |
| Threshold | 0.95 | Neighbor density and contract evidence |
| Parallelism | p=2 | Runtime profile and metrics view |
| Generator | selected backend | Contract-to-answer handoff |

The snapshot scenario shows why the bounded window matters. When related reports arrive from multiple sources, the system connects them into a single family and exports a compact contract with source coverage and neighbor identifiers. The UI grounds the generated summary in event identifiers, nearest-neighbor rows, and emitted join pairs visible to the audience.

The runtime-inspection scenario shows how the same event is visible at two levels. At the outer level, the application pipeline shows the control path from event to generator. At the inner level, the runtime view shows the operator path that owns the semantic state.

The generator scenario shows the contract reaching the model and the answer returning to the same evidence rows. The audience can inspect the contract first, then compare it with the generated response and token/latency metadata. A provider-agnostic display keeps this stage reusable across generator backends while preserving the main claim: the generator consumes bounded evidence, and evidence identifiers can be traced back to the runtime state.

## 4 Prepared Evidence

The narrated walkthrough is intentionally small enough for a live audience. To show more than a toy sample, the prepared evidence

| Window | Parallelism | Throughput | Speedup | Pairs |
|---|---|---|---|---|
| 512 | 1 | 22.7k ev/s | 1.00× | 6,480 |
| 512 | 2 | 26.4k ev/s | 1.16× | 8,450 |
| 1024 | 1 | 18.0k ev/s | 1.00× | 9,025 |
| 1024 | 2 | 28.1k ev/s | 1.57× | 7,260 |

| Gen. run | Ev. | Ans. | FB | LLM p50/p95 |
|---|---|---|---|---|
| Contract-to-answer | 20 | 20 | 0 | 8.12/11.01s |

**Table 2: Prepared evidence from real runs. Top: clustered runtime replay on 3,000 NVD records with BGE embeddings, using the p=1 baseline and the p=2 operating point used in the live profile. Bottom: a 20-event contract-to-answer generator run reports answer coverage, fallback count, and LLM latency.**

uses 3,000 public NVD records from 2024-Q1. Embeddings are produced by `BAAI/bge-small-en-v1.5` through SentenceTransformers, normalized to 384 dimensions, sharded for artifact storage, and replayed from cache so the runtime experiment is deterministic. A sampled vector was recomputed from the original text and matched the cache within floating-point roundoff, which guards against reporting placeholder vectors.

## 4.1 Workloads and Metrics

The runtime workload contains 3,000 NVD records and 6,000 vector insertions because each event is replayed into the two input sides of the join. We report feed-and-drain latency, event throughput, speedup relative to one worker at the same window size, and emitted pair count for p=1 and p=2. The prepared runtime run uses `clustered_join`, a low similarity threshold of 0.5, multicast fanout two, and sequence timestamps so window sizes correspond to recent vector arrivals rather than calendar time. The generator workload replays 20 public vulnerability events through the same contract layer and reports generated answers, fallback count, and LLM p50/p95 latency separately from runtime throughput.

At a 512-arrival window, Table 2 reports throughput rising from 22.7k events/s at p=1 to 26.4k events/s at p=2. At 1,024 arrivals, throughput rises from 18.0k events/s to 28.1k events/s, a 1.57× speedup. We do not claim linear scaling; p=4 and beyond are out of scope for this demo paper and will be reported separately. The p=2 clustered profile is therefore the default parallel operating point used in the console metrics view. The generator run completes all 20 answers without fallback, and each answer carries the contract identifiers it consumed, so the audience can trace generated text back to specific retained events. Together, these results show runtime behavior at stream scale and contract-to-answer behavior at demo scale.

## 4.2 What the Evidence Shows

The prepared evidence covers three things the live walkthrough cannot show on its own. The live path has enough data to show real behavior: reports from multiple source classes produce incident families, cross-source clusters, and both correlation and novelty contracts. The runtime boundary remains measurable on a larger public stream: the same cached embeddings and runtime API process thousands of records and expose throughput, latency, and emitted pairs. The contract handoff also remains inspectable when generation is enabled: answer coverage, fallback count, and LLM latency are reported next to the runtime state. A demo attendee does not need to infer behavior from a hidden log; the transition is visible through retained records, match pairs, clusters, exported contracts, and the final generated answer.

# 5 Discussion and Conclusion

BriskSnapshot is a demonstration of a system boundary: semantic-window state remains inside a persistent vector runtime, while the application receives bounded contracts that are stable enough to inspect, log, route, and summarize. The contribution is making streaming vector evidence observable as application behavior, rather than improving similarity search itself. The demo therefore emphasizes the handoff from event stream to runtime evidence to contract output.

There are two takeaways for attendees. The first is operational: a semantic window should expose enough state for a human to understand why an application-level decision changed. In BriskSnapshot, the presenter can point from a contract back to retained events and join pairs, which makes the system easier to debug than a pipeline that only displays generated text. The second takeaway is architectural: keeping the join state inside a long-lived runtime avoids rebuilding semantic context at every application step, and lets the audience see freshness, boundedness, and contract emission as one continuous behavior.

# References


[1] Tyler Akidau, Alex Balikov, Kaya Bekiroğlu, Slava Chernyak, Josh Haberman, Reuven Lax, Sam McVeety, Daniel Mills, Paul Nordstrom, Sam Whittle, Robert Wilfong, and Lukasz Zając. 2015. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1792–1803.

[2] Anonymous. 2026. SAGE: A Dataflow-Native Framework for Modular, Controllable, and Transparent LLM-Augmented Reasoning. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*. PMLR. Author list withheld for double-anonymous review.

[3] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. 2015. Apache Flink: Stream and Batch Processing in a Single Engine. *IEEE Data Engineering Bulletin* 38, 4 (2015), 28–38.

[4] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. Retrieval Augmented Language Model Pre-Training. In *Proceedings of the 37th International Conference on Machine Learning*. PMLR, Virtual Event, 3929–3938.

[5] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.

[6] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.

[7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., Red Hook, NY, USA, 9459–9474.

[8] Duo Lu, Siming Feng, Jonathan Zhou, Franco Solleza, Malte Schwarzkopf, and Uğur Çetintemel. 2025. VectraFlow: Integrating Vectors into Stream Processing. In *Proceedings of the 15th Annual Conference on Innovative Data Systems Research (CIDR)*. Amsterdam, The Netherlands.

[9] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.

[10] Murugan Sankaradas, Ravi K. Rajendran, and Srimat T. Chakradhar. 2024. StreamingRAG: Real-time Contextual Retrieval and Generation Framework. In *Proceedings of the Workshop on AI for Systems (AI4Sys), co-located with HPDC*. ACM, Pisa, Italy, 1–6. doi:10.1145/3660605.3660943